



# Online Chat 接口链路文档

系统: online-chat (Spring Cloud 微服务 + Vue3 SPA) 仓库: <https://github.com/liugl951127/chatInfo> 当前版本: cae36ef (main) 端口约定: cs-auth=9001, cs-gateway=9000, cs-im=9002

## 0. 架构概览



前后端契约: 后端返回 {code, message, data} 三元组；code=0 成功，data 为业务载荷。axios 拦截器已统一解包。

## 1. 鉴权模块 (cs-auth)

| 接口 | 方法   | 路径             | 含义              | 实现原理                                                                      |
|----|------|----------------|-----------------|---------------------------------------------------------------------------|
| 登录 | POST | /auth/login    | 用户名密码登录, 返回 JWT | bcrypt.matches(rawPwd, dbHash) → JWT(HS512) 签发, claims: uid/role/nickname |
| 注册 | POST | /auth/register | 新用户注册           | 同上, 默认 role=CUSTOMER                                                      |

|       |      |                                    |                      |
|-------|------|------------------------------------|----------------------|
| 注册管理员 | POST | /auth/register-admin(开发用) 创建 admin | 同上, 强制<br>role=ADMIN |
|-------|------|------------------------------------|----------------------|

前端调用链:

```
Login.vue
↓ axios.post('/auth/login', {username, password})
↓ axios data → userStore.set({token, role, ...})
↓ router.replace(role==='AGENT' ? '/agent' : '/customer')
```

JWT 透传机制:

```
cs-gateway
↓ AuthGlobalFilter JWT, mutate header X-User-Id
↓ cs-im / cs-auth
↓ JwtAuthInterceptor JWT (ThreadLocal)
↓ → UserContext.set(uid, username, role, nickname) (ThreadLocal)
↓ Controller UserContext.userId()
```

Token 生命周期: 默认 24h 过期, 不在客户端做刷新 (简化)。前端用 Pinia useUserStore.token 持久化到 localStorage。

## 2. 会话模块 (cs-im /api/im/session/\*\*)

| 接口                                       | 方法   | 含义                        | 角色             |
|------------------------------------------|------|---------------------------|----------------|
| /mine                                    | GET  | 我的会话列表 (含 peerOnline)     | 任意已登录          |
| /create?skill=                           | POST | 创建会话, 自动分配坐席              | CUSTOMER       |
| /claim                                   | POST | 抢单 (从等待队列取一个)             | AGENT          |
| /waiting                                 | GET  | 查看等待队列 (admin)            | AGENT/ADMIN    |
| /[{id}]/transfer?toAgentId=&reason=      | POST | 转接到其他坐席                   | AGENT          |
| /[{id}]/request-transfer?preferredSkill= | POST | 客户申请转接客服                  | CUSTOMER       |
| /[{id}]/rate?rating=&comment=            | POST | CSAT 评分                   | CUSTOMER       |
| /[{id}]/close                            | POST | 关闭会话 (会触发 CSAT 弹窗)        | AGENT/CUSTOMER |
| /[{id}]/exit?reason=                     | POST | 客户主动退出 (不弹 CSAT)          | CUSTOMER       |
| /bootstrap                               | GET  | 启动面板 (待接/已接/未读总数)         | AGENT          |
| /offline/drain                           | GET  | 拉取离线消息 (WebSocket 失联期间堆积) | 任意             |
| /[{id}]/unread                           | GET  | 该会话的未读数                   | 任意             |
| /agent/status?status=                    | POST | 切换坐席状态 (ONLINE/BUSY/AWAY) | AGENT          |



```

    ■■ auditLog: action=CUSTOMER_TRANSFER, detail=from=+to=+skill=
↓
Customer.vue onEvent(TRANSFERRED) → refreshSessionFromServer (■■ agentId)
Agent.vue      onEvent(TRANSFERRED) → refreshSessions (■■■■■)

```

## 2.4 坐席转接 (POST /api/im/session/{id}/transfer) — 区别于客户转接

坐席主动转接到指定坐席 (指定 toAgentId), 客户转接是系统自动选择。两者的 WebSocket 事件都是 TRANSFERRED, 推送目标相同。

# 3. 消息模块 (cs-im /api/im/message/\*\* + WebSocket)

## 3.1 REST 接口

| 接口                                     | 方法   | 含义             |
|----------------------------------------|------|----------------|
| /api/im/session/{id}/messages?limit=1  | GET  | 历史消息分页         |
| /api/im/session/message/{msgId}/recall | POST | 撤回一条消息 (2 分钟内) |
| /api/im/session/message/{msgId}/read   | POST | 单条已读回执         |
| /api/im/session/{id}/read-all          | POST | 整会话已读          |

## 3.2 WebSocket STOMP 协议

### 3.2.1 端点

| Endpoint     | 用途    | 携带 token                      |
|--------------|-------|-------------------------------|
| /ws/customer | 客户连接  | ?token=xxx (浏览器 WS header 限制) |
| /ws/agent    | 坐席连接  | 同上                            |
| /ws/admin    | 管理员连接 | 同上                            |

Handshake 拦截链 (HandshakeAuthInterceptor):

```

HTTP Upgrade WebSocket
↓ ■■ token: header Authorization → query ?token=
↓ JwtUtil.parse(secret) → uid/role/nickname
↓ ■■ ServerHttpRequest.attributes (■ STOMP CONNECT ■■)
↓ WS upgrade ■■

```

STOMP CONNECT 帧 (StompAuthChannelInterceptor.preSend):

```

■■■■ CONNECT
↓ preSend ■■
↓ ■ SessionAttributes ■ user ■■, ■■■ StompPrincipal
↓ accessor.setUser(principal)
↓ presenceService.online(uid, role) ← ■■■ Redis + ■■ Set
↓ ■■■■ → ■■ PresenceChangedEvent
↓ WsPushService.@EventListener → ■■■■ → ■■■ PRESENCE=ON

```

```
↓ ■■■ broker
↓ ■■■■ CONNECTED ■
```

STOMP DISCONNECT 帧:

```
↓ preSend ■■, ■ principal
↓ presenceService.offline(uid, role)
↓ localRegistered.remove(uid) → ■■ PresenceChangedEvent(online=false)
↓ ■■■ PRESENCE=OFF
```

### 3.2.2 订阅目的地 (Destination)

| Destination                    | 类型               | 谁订阅      | 内容                                       |
|--------------------------------|------------------|----------|------------------------------------------|
| /user/queue/messages           | user-destination | 登录后的所有用户 | 聊天消息推送 (MessageDTO)                      |
| /user/queue/events             | user-destination | 同上       | 事件 (READ/RECALL/TRANSFERRED/CLOSED/ECHO) |
| /topic/sessions/new            | topic            | 坐席       | 新会话等待通知 (NEW_WAITING)                    |
| /topic/typing/{sessionId}topic |                  | 会话双方     | 输入状态 (TYPING)                            |

### 3.2.3 应用层 @MessageMapping

| 路径                      | 处理方法                                          | 入参                     | 行为                                                |
|-------------------------|-----------------------------------------------|------------------------|---------------------------------------------------|
| /app/send/{sessionId}   | MessageController.send                        | MessageDTO + Principal | 存消息 + 推双方 (peer 推 /queue/messages, sender 推 echo) |
| /app/typing/{sessionId} | MessageController.typing {typing: true/false} |                        | 推 /topic/typing/{sessionId} 给会话旁观方                |

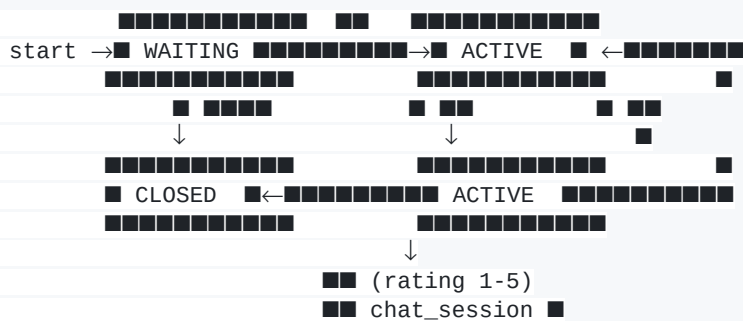
消息发送链路 (核心):

```
Customer ■■■■
↓ Customer.vue send()
↓ stomp.publish('/app/send/'+sessionId, {sessionId, msgType, content})
↓ STOMP SEND ■■ cs-gateway → cs-im
↓ MessageController.send()
  ■■ @DestinationVariable sessionId ■■ (URL ■■)
  ■■ MessageService.handleIncoming()
  ■ ■■ DB ■■ chat_message
  ■ ■■ session.lastMessage ■■■■
  ■ ■■ ■■■■ echo: convertAndSendToUser(senderId, "/queue/messages", dto)
  ■ ■■ ■■■■ (peer):
  ■ ■ ■■ unreadCounterService.incr(peerId, sessionId) (Redis HASH)
  ■ ■ ■■ if presenceService.isOnline(peerId):
  ■ ■ ■   convertAndSendToUser(peerId, "/queue/messages", dto) ← ■■■■
  ■ ■ ■■ else:
  ■ ■   offlineStore.push(peerId, dto) ← ■■■■
  ■■ ■■■■ frame, onIncomingMessage ■■ appendMessage
```

接收端渲染 (Customer.vue):

```
ws-client.js._doSubscribe callback (raw STOMP frame)
↓ try JSON.parse(msg.body)
↓ onIncomingMessage(msg)
  ■■ if session.value && msg.sessionId && ■■■ → ■■ (■■■■■■■■)
  ■■ appendMessage(msg) → messages.value.push(msg) → Vue ■■■■■ DOM ■■
  ■■ if ■■■■■ → imApi.readMessage(msg.id) ■■■■
```

#### 4. 客服状态机



坐席在线状态 (Redis key): - chat:agent:online (Set) — 在线坐席 id 集合 - chat:agent:status:{id} (String) — ONLINE/BUSY/AWAY, TTL=30min - chat:agent:session:{id} (String) — 坐席当前会话 id, 用于 isBusy 判断 - chat:user:online:{id} (String) — 全员在线标记, TTL=30min

## 5. 在线状态实时同步 (PresenceService) — v2.0.1 新增

## 5.1 数据流

```

##### (WS CONNECT)
↓ PresenceService.online(uid, role)
■■ localRegistered.add(uid) ← #####
■   ■■ true (■■) / ■■ false (■■)
■■ Redis SET chat:user:online:{uid} = 1 (TTL=30min)
■■ if AGENT:
■   ■■ SADD chat:agent:online {uid}
■   ■■ ■■: AgentStatusService.setStatus(uid, ONLINE)
■■ ■■ → eventPublisher.publishEvent(PresenceChangedEvent online=true)

#####:
↓ @EventListener WsPushService.onPresenceChanged
↓ ■■■ uid ##### (status IN WAITING/ACTIVE)
↓ ■■■ peer (■■/■■): pushToUser(peerId, {type:'PRESENCE', online:true, ...})

```

用户下线 (WS DISCONNECT):

```
PresenceService.offline(uid, role)
■■ localRegistered.remove(uid) ← ■■■■■■■■■■■■■■■■■■■■
■■ Redis DEL chat:user:online:{uid}
■■ if AGENT: SREM chat:agent:online, DEL status key
■■ ■■■■■ → ■■ PRESENCE=OFF
```

## 5.2 循环依赖的解法

PresenceService → WsPushService → SimpMessagingTemplate → WebSocketConfig → StompAuthChannelInterceptor → PresenceService

用 Spring ApplicationEvent 解耦: PresenceService 只发事件, 不依赖 WsPushService。

### 5.3 前端渲染

Agent.vue:

```
onEvent({type:'PRESENCE', sessionId, online, ts})
  ■■■ sessions[] ■■■■ sessionId, ■■ peerOnline + lastSeen
  ■■ ■■■■■■ current, ■■■ current.peerOnline
  ■■ UI ■■■■■ → ■■■■■■■■■■/■, ■■ "■■■■■" tag
```

初值 (避免刷新时空白): GET /api/im/session/mine 返回 SessionView.peerOnline (后端查 Redis)。

## 6. 录制回溯 (cs-im /api/im/record/\*\*)

### 6.1 数据模型

```
chat_record (
  id, session_id, user_id, user_role,
  started_at, ended_at, end_reason,      -- NORMAL/USER_STOP/PAGE_CLOSE/PROCESS_KILLED/ERROR
  chunk_count, total_bytes,
  consent_given,                        -- ■■: ■■■■■■■■■■
  created_at
)

chat_record_chunk (
  id, record_id, sequence_no,           -- ■■■■ (■ 0 ■■)
  mime_type ('video/webm'),
  duration_ms, byte_size,
  storage_path,                        -- <root>/<recordId>/<seq>-<uuid>.webm
  uploaded_at
)

chat_audit_log (id, actor_id, actor_role, action, target, detail, ip, user_agent, created_at)
```

### 6.2 REST 接口

| 接口                                       | 方法   | 含义                          | 权限                       |
|------------------------------------------|------|-----------------------------|--------------------------|
| /init?sessionId=&consent=                | POST | 开启录制 (consent 必须 true)      | 任意已登录                    |
| /chunk?recordId=&sequenceNo=&durationMs= | POST | 上传一个分片 (multipart)          | 仅 record.userId          |
| /end?recordId=&endReason=                | POST | 结束录制                        | 仅 record.userId          |
| /session/{id}                            | GET  | 列出会话所有录像                    | 任意已登录                    |
| /session/{id}/with-chunks                | GET  | 同上 +<br>每个录像的分片列表<br>(一次拉完) | 三道权限<br>(本人/该会话坐席/admin) |
| {id}/chunks                              | GET  | 录像的所有分片                     | 任意已登录                    |



|                 |     |                     |      |
|-----------------|-----|---------------------|------|
| /chunk/{id}/raw | GET | 下载一个分片的二进制<br>(回放用) | 三道权限 |
|-----------------|-----|---------------------|------|

### 6.3 前端 SDK — record-sdk.js

#### 6.3.1 架构

ChatRecordSDK  
■ ■ opts: apiBase, token, sessionId, userId, nickname, target, fps, chunkDurationMs, bitrate, watermark  
■ ■ ■: html2canvas ■ ■ ■ ■ DOM → offscreen canvas → MediaRecorder → WebM blob  
■ ■ ■: FormData multipart → /api/im/record/chunk (■ ■ ■ ■ 3 ■)

#### 6.3.2 录制流程

```
Customer.vue tryRecorder() (■ ■ ■ ■ ■)
↓ new ChatRecordSDK({...})
↓ sdk.start()
  ■ ■ 1. ■ ■ init (consent=true)
  ■   ↓ chat_record INSERT (consentGiven=true)
  ■   ↓ audit_log: RECORD_INIT
  ■   → ■ ■ recordId
  ■ ■ 2. ■ ■ offscreen canvas (■ ■ 1280x800)
  ■ ■ 3. setInterval 1000/fps ms → html2canvas(target) → drawImage ■ canvas
  ■   (■ ■ ■ ■ ■ ■ ■ ■, ■ warn)
  ■   if watermark: _drawWatermark(ctx)
  ■     ■ ■ ■ ■ ■: "■ ■ ■ ■ ■ ■ ■ ■ - ■ ■ ■ ■ ■ ■ ■ ■"
  ■     ■ ■ ■ ■ ■: ■ ■ ■ / ■ ■ #ID / ■ ■ #ID / ■ ■ #ID
  ■ ■ 4. canvas.captureStream(fps) → MediaRecorder (WebM VP9/VP8, 500kbps)
  ■ ■ 5. recorder.start(5000) ← ■ 5s ■ ■ ■ ■ ■
  ■   ondataavailable(blob):
  ■     ■ ■ chunkQueue.push({blob, sequence, duration, uploaded:false})
  ■     ■ ■ uploadPromise.then(() => _uploadOne(blob, sequence))
  ■     (■ ■ ■ ■ ■, ■ ■ ■ ■ ■)
  ■ ■ 6. ■ visible indicator + ■ ■ beforeunload/pagehide/visibilitychange
```

#### 6.3.3 异常容错

| 场景               | 处理                                                                                       |
|------------------|------------------------------------------------------------------------------------------|
| 客户主动退出           | Customer.customerExit() → stopRecorder('USER_STOP') → 等所有分片上传 → /end?endReason=USER_STOP |
| 客户切换页面前          | beforeunload/pagehide 事件 → _onUnload() → navigator.sendBeacon() 把剩余分片 + end 一次性同步发出      |
| 手机切后台            | visibilitychange=hidden → 同上 sendBeacon 兜底                                               |
| 杀进程              | best-effort: in-memory 未上传分片丢失, 但周期性上传 (5s/段) 保证多数内容已落库。audit_log 留痕                     |
| 网络抖动             | _uploadOne 失败重试 3 次, 指数退避 500ms→1s→1.5s                                                  |
| html2canvas 单帧失败 | warn + 跳过, 继续录制                                                                          |

#### 6.3.4 静默模式 (低摩擦版) — vs 标准模式





## 7.2 loadHistory VS drainOffline

| 路径                                          | 触发                      | 数据来源                          |
|---------------------------------------------|-------------------------|-------------------------------|
| GET /api/im/session/{id}/messages?limit=100 | onReconnected, 组件 mount | MySQL chat_message 表 (权威, 不丢) |
| GET /api/im/session/offline/drain           | 组件 mount 时 (一次性)        | Redis List (实时, 但可能 LTRIM 截断) |

策略: mount 时同时拉两个, 先 history 再 merge offline (按 id 去重)。reconnect 时只拉 history。

## 8. 前端 STOMP 客户端 (ws-client.js)

### 8.1 关键设计

```
class StompClient {
  subscribe(dest, cb) // dest: dest, cb: callback
  connect(path)        // @stomp/stompjs v7 client.activate()
                        // path: '/ws/customer' | '/ws/agent'
}
```

幂等订阅 解决 v2.0.0 发现的 STOMP 消息双发 bug:

```
[bug] UI onConnected subscribe + onConnect subscribed Set forEach
      → destination 2 → STOMP frame 2 →

[fix]
1. subscribe() dest: dest, cb: callback
2. onConnected (firstConnected), dest, cb
3. subscribed Set dest, UI
4. UI connect()
```

### 8.2 重连机制

```
WS (dest/path)
↓ onWebSocketClose
↓ _intentionallyClosed=false →
↓ reconnectDelay = min(32s, 1000 * 2^reconnectAttempt)
↓ → onConnect
↓ subscribed.forEach →
↓ onReconnected (onConnected)
  ↓ Customer.vue → loadHistory
  ↓ Agent.vue → refreshSessions + readAll
```

## 9. 错误码约定

| code | 含义 | 前端处理 |
|------|----|------|
|------|----|------|

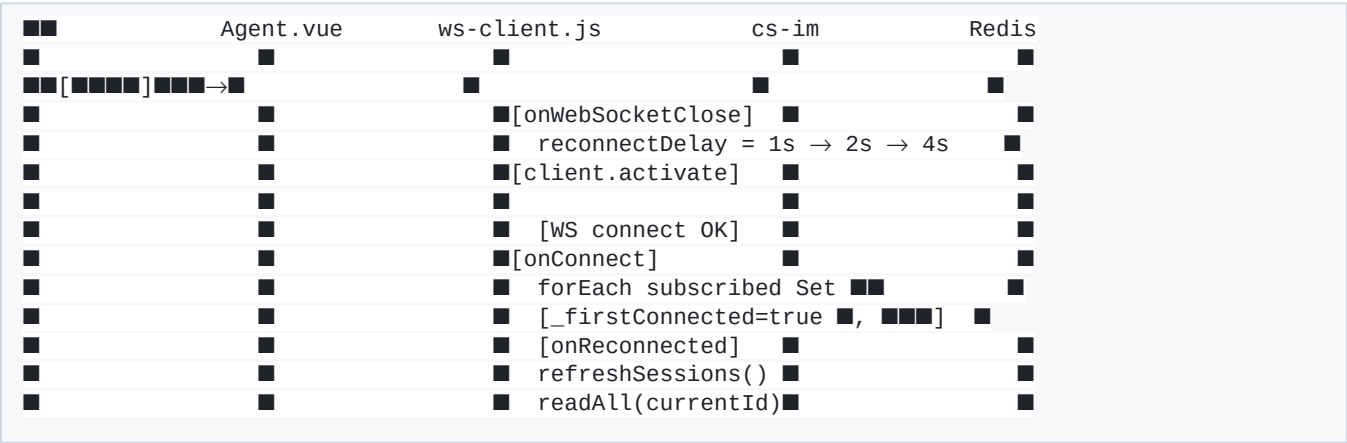
|     |                 |                       |
|-----|-----------------|-----------------------|
| 0   | 成功              | 正常                    |
| 401 | 未登录 / token 失效  | 清 userStore, 跳 /login |
| 403 | 无权 (越权访问)       | ElMessage.error, 不重定向 |
| 404 | 资源不存在           | ElMessage.error       |
| 409 | 状态冲突 (重复操作/已关闭) | 幂等返回, UI 提示 "已处理"     |
| 500 | 内部错误            | ElMessage.error, 上报   |

## 10. 关键时序图

### 10.1 客户接入会话完整流程



### 10.2 坐席断网 → 重连 → 补漏



## 11. 部署清单

| 组件         | 端口   | 启动顺序 | 关键配置                                                            |
|------------|------|------|-----------------------------------------------------------------|
| MariaDB    | 3306 | 1    | online_chat 库,<br>root/951127, schema 见<br>chat_record.sql      |
| Redis      | 6379 | 1    | 无密码, 用于 presence/ws<br>pubsub/offline store                     |
| cs-auth    | 9001 | 2    | JWT_SECRET<br>必须三服务一致<br>(cs-auth/cs-gateway/cs-im)             |
| cs-im      | 9002 | 3    | record.storage-path=/tmp/chat-r<br>multipart.max-file-size=50MB |
| cs-gateway | 9000 | 4    | 静态路由 /auth/**<br>/api/im/** /ws/**                              |
| Vite dev   | 5173 | 5    | host: '0.0.0.0', proxy<br>/auth /api /ws →<br>localhost:9000    |

生产部署要点: - 用 nginx 替代 Vite (静态文件 + 反代 /api/im/ws 到 cs-gateway) - nginx 配置 WebSocket 升级:  
proxy\_set\_header Upgrade \$http\_upgrade; proxy\_set\_header Connection "upgrade"; - JWT secret 用环境变量,  
不要硬编码 - chat\_record 文件存储换成对象存储 (S3/MinIO) 或配 ffmpeg 合流 - 30 天 cron 清理过期录像: DELETE FROM  
chat\_record WHERE ended\_at < NOW() - 30 DAY

## 12. 待优化 / 已知限制

| 项          | 现状                      | 改进方向                                     |
|------------|-------------------------|------------------------------------------|
| WebM 直拼回放  | 首段可播, 后续分段可能掉帧          | MSE 流式 appendBuffer 或服务端 ffmpeg<br>remux |
| 录制体积       | 500kbps × 1h ≈ 220MB    | 客户用 200kbps, 坐席用 500kbps 分档              |
| 录制过期清理     | 仅标 ended_at, 无 cron     | 加 @Scheduled 任务                          |
| 客户录像重连断点   | 进程被杀时丢分片                | IndexedDB 暂存 in-memory 分片                |
| 客户转接无新会话   | in-place 切换 agent, 历史保留 | 已经做了, 但 skill 切换能力有限                     |
| 多坐席场景下用户感知 | 只能从头像判断换了客服             | 加 "上一位客服" 字段展示                           |